`tctl_sat`— User Manual

A tableau decision procedure for TCTL satisfiability

`tctl_sat` answers one question: given a TCTL formula, *is there a timed automaton that satisfies it?* On success it can also hand back such an automaton as a certificate. This manual covers the syntax it accepts, how to run it, and what it produces.

Contents

1 Getting started **1**
- 1.1 Building . . . 1
- 1.2 A first check . . . 2
- 1.3 What the answers mean . . . 2

2 Syntax **2**
- 2.1 Grammar . . . 2
- 2.2 Operators . . . 2
- 2.3 Intervals . . . 3
- 2.4 Arithmetic leaves . . . 3

3 Running the tool **3**
- 3.1 Input . . . 3
- 3.2 Options . . . 4
- 3.3 Environment flags . . . 4

4 Examples **4**
- 4.1 Reachability and safety in conflict . . . 4
- 4.2 Strictness matters . . . 4
- 4.3 A leads-to specification . . . 4
- 4.4 Nested timed modalities . . . 5
- 4.5 A contradiction the tool must catch . . . 5

5 Witness automata **5**
- 5.1 Reading the output . . . 5
- 5.2 What is guaranteed . . . 6
- 5.3 Limits . . . 6

1 Getting started

1.1 Building

`tctl_sat` needs a C++20 compiler, CMake $\geq$ 3.20, Z3, and `git`.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

That is the whole process. The zone layer (HDBM) is header-only and is cloned automatically on the first configure if it is not already present. Z3 is the one external library and comes from the system: `brew install z3` on macOS, `apt install z3 libz3-dev` on Linux.

1.2 A first check

```
$ ./build/tctl_sat 'EF[1,2] p'
1: SATISFIABLE
```

Each answer is `SATISFIABLE`, `UNSATISFIABLE` or `TIMEOUT`, prefixed by the input's line number.

1.3 What the answers mean

The procedure is *sound* and *terminating* for all of TCTL: a `SATISFIABLE` verdict always means a witness automaton exists. It is *complete* over the non-punctual fragment — every interval $[a, b]$ with $a < b$ — and also at temporal nesting depth one even with punctual intervals $[a, a]$.

Outside that, a punctual interval nested inside another timed modality is accepted and warned about:

```
$ ./build/tctl_sat 'EG[1,5] (EF[2,2] p)'
1: warning: punctual interval [a,a] nested inside another timed modality ...
1: SATISFIABLE
```

There, `SATISFIABLE` is still reliable; `UNSATISFIABLE` may be a false negative.

2 Syntax

2.1 Grammar

$$
\begin{array}{rcl}
\varphi & ::= & \texttt{true} \mid \texttt{false} \mid p \mid \texttt{\{}\ \mathit{arith}\ \texttt{\}} \\
& \mid & \texttt{!}\varphi \mid \varphi\ \texttt{\&}\ \varphi \mid \varphi\ \texttt{|}\ \varphi \mid \varphi\ \texttt{->}\ \varphi \mid \varphi\ \texttt{<->}\ \varphi \\
& \mid & \texttt{EF}I\ \varphi \mid \texttt{AF}I\ \varphi \mid \texttt{EG}I\ \varphi \mid \texttt{AG}I\ \varphi \\
& \mid & \texttt{E(}\varphi\ \texttt{U}I\ \varphi\texttt{)} \mid \texttt{A(}\varphi\ \texttt{U}I\ \varphi\texttt{)} \\
& \mid & \texttt{E(}\varphi\ \texttt{R}I\ \varphi\texttt{)} \mid \texttt{A(}\varphi\ \texttt{R}I\ \varphi\texttt{)} \\
I & ::= & \varepsilon \mid \texttt{[}a\texttt{,}b\texttt{]} \mid \texttt{(}a\texttt{,}b\texttt{)} \mid \texttt{[}a\texttt{,}b\texttt{)} \mid \texttt{(}a\texttt{,}b\texttt{]} \mid \texttt{[}a\texttt{,inf)}
\end{array}
$$

Atoms are identifiers such as `p`, `q`, `p1`. An omitted interval means $[0, \infty)$, so `EF p` and `EF[0,inf) p` are the same formula.

2.2 Operators

Written	Means	Timed form	Reading
`EF` φ	$\exists\Diamond$	`EF[1,5] p`	some run reaches p within $[1, 5]$
`AF` φ	$\forall\Diamond$	`AF[0,3] p`	every run reaches p within $[0, 3]$
`EG` φ	$\exists\Box$	`EG[1,5] p`	some run keeps p throughout $[1, 5]$
`AG` φ	$\forall\Box$	`AG[0,2] p`	every run keeps p throughout $[0, 2]$
`E(`φ `U` ψ`)`	$\exists\,\mathbf{U}$	`E(p U[1,3] q)`	some run holds p until q, in $[1, 3]$
`A(`φ `R` ψ`)`	$\forall\,\mathbf{R}$	`A(p R[2,5] q)`	ψ holds until released by φ

There is no next operator. TCTL has none, and `EX`/`AX` are rejected by the parser. They exist only as internal markers inside the construction.

2.3 Intervals

Bounds are non-negative integers. Both closed and open endpoints are supported, and the strictness is respected exactly: `EF[1,2] p` and `EF(1,2) p` are different questions.

A *punctual* interval $[a, a]$ is accepted — see the completeness note in Section 2's preceding chapter. An empty interval (a lower bound above the upper one, or an open singleton) is a parse error.

2.4 Arithmetic leaves

Constraints in braces are handed to Z3 rather than to the temporal machinery:

```
{x <= 5} & {x > 10} % unsatisfiable: no such x
EF ({x <= 10} & p) % satisfiable
```

Variables are integer-sorted by default; `-real-vars x,y` makes the named ones reals.

3 Running the tool

3.1 Input

A formula can be given directly, or in a file with one formula per line:

```
./build/tctl_sat 'AG[0,2] p' # a single formula
./build/tctl_sat properties.txt # a file of them
```

In a file, blank lines are skipped and everything after `#` is a comment. Each formula is answered independently, prefixed by its line number:

```
# properties.txt -- a small specification
EF[1,3] request # a request is possible in [1,3]
AG (request -> AF[0,5] grant) # every request is granted within 5
AG[0,10] !error # no error in the first 10 units
```

```
$ ./build/tctl_sat properties.txt
1: SATISFIABLE
2: SATISFIABLE
3: SATISFIABLE
```

Answering each line separately asks whether each property is satisfiable *on its own*. To ask whether they are *jointly* satisfiable — the usual specification sanity-check — conjoin them with `-tot`:

```
$ ./build/tctl_sat --tot properties.txt
1: SATISFIABLE
```

This is how an inconsistent specification is caught. Each of the following is satisfiable alone, but not together:

```
$ printf 'EF[1,2] p\nAG[0,3] !p\n' > bad.txt
$ ./build/tctl_sat bad.txt
1: SATISFIABLE
2: SATISFIABLE
$ ./build/tctl_sat --tot bad.txt
1: UNSATISFIABLE
```

3.2 Options

Option	Effect
`-tot`, `-all`	conjoin every input formula and decide the conjunction
`-model`	extract a witness automaton (Section 5)
`-stats`	report the number of tableau nodes explored
`-output <dir>`	directory for extracted witnesses
`-real-vars a,b`	treat the named arithmetic variables as reals
`-selftest`	run the built-in semantic test suite
`-help`	full option list

3.3 Environment flags

Variable	Effect
`TCTL_DEBUG=1`	trace every expanded node and each component, to stderr
`TCTL_LEGACY_LOOP=1`	use the earlier path-local loop acceptance (comparison only)

4 Examples

Each example below was run with the version of `tctl_sat` this manual ships with.

4.1 Reachability and safety in conflict

```
$ ./build/tctl_sat 'EF[1,2] q & AG[0,3] !q'
1: UNSATISFIABLE
```

The first conjunct demands q somewhere in $[1, 2]$; the second forbids it everywhere in $[0, 3]$. Since $[1, 2] \subseteq [0, 3]$ there is no model. Widen the safety window so the two no longer overlap and a model appears:

```
$ ./build/tctl_sat 'EF[2,2] q & AG[0,1] !q'
1: SATISFIABLE
```

4.2 Strictness matters

```
$ printf 'EF[1,2] p & AG[0,1] !p\nEF(1,2) p & AG[0,1] !p\n' > s.txt
$ ./build/tctl_sat s.txt
1: SATISFIABLE
2: SATISFIABLE
```

Both hold: p can be placed strictly after 1. The zone layer tracks the open/closed endpoint exactly rather than rounding it to an integer.

4.3 A leads-to specification

The classic response property, `AG(request -> AF[0,5] grant)`, is the shape most often written for Uppaal. It is satisfiable on its own, and stays so together with a bounded-response requirement:

```
$ ./build/tctl_sat --tot leadsto.txt
1: SATISFIABLE
```

where `leadsto.txt` contains

```
AG (request -> AF[0,5] grant)
AG[0,20] !error
EF[1,3] request
```

4.4 Nested timed modalities

Nesting is where the procedure does real work. The following is satisfiable, and the node count shows the cost:

```
$ ./build/tctl_sat --stats 'EG[1,5] (EF[2,4] (EG[6,8] p))'
1: SATISFIABLE
  Stats: nodes=261
```

4.5 A contradiction the tool must catch

```
$ ./build/tctl_sat 'EF p & !EF p'
1: UNSATISFIABLE
```

5 Witness automata

A `SATISFIABLE` verdict is a claim that a model exists. With `-model`, `tctl_sat` hands the model over as well, so the claim can be inspected or checked by another tool.

```
$ ./build/tctl_sat --model 'EF[1,2] p'
1: SATISFIABLE
  Witness: 5 locations, 6 transitions, 2 clocks
  wrote witness_1.{dot,json,xml}
```

Three files are written per satisfiable formula: Graphviz `.dot` for viewing, `.json` for further processing, and `.xml` in Uppaal's format.

5.1 Reading the output

For `EF[1,2] p` the extracted automaton is

```
L0 -> L1 [x_1 < 1]
L1 -> L3 [x_1 <= 1 && x_1 >= 1]
L3 -> L4 [x_1 >= 1]
L4 -> L4 [x_1 >= 1]
L0 -> L2 ; L2 -> L2
```

with `p` holding at `L3`. Read left to right: let the clock run below 1, step onto $x_1 = 1$ — inside the window $[1, 2]$ — where p holds, then continue on a self-loop so that time diverges. The `L0`–`L2` branch is the other half of the cover rule, where the obligation is discharged rather than pursued.

The self-loop at the end matters: a witness must let time diverge, or the run would be Zeno and the semantics would not inspect it.

5.2 What is guaranteed

Locations are the marked nodes, labelled by the atoms they assert and carrying the deadline of any universal until as an invariant. Transitions are the retained discrete successors, guarded by the target's zone, resetting a formula-clock exactly where its modality is launched. Delay contributes no transition: time simply advances inside a location.

Every extracted automaton is checked before it is written — that it is non-blocking, that no location asserts both a proposition and its negation, and that guards only read declared clocks. A failure is reported as `[INVALID]` together with the reason.

5.3 Limits

Extraction is a property of the component-based acceptance engine, which is the default, so it needs no extra flag. It is the *legacy* engine that cannot do it: `TCTL_LEGACY_LOOP=1` returns a bare verdict and never builds a marking, so there is nothing to read an automaton off. The zone recorded for a node is the $\cong_M$-canonical one, so guards are read off the extrapolated zone rather than the raw one — the same zone the procedure itself compares on.